

BACCALAURÉAT GÉNÉRAL

Sujet 02- Session 2026

NUMERIQUE ET SCIENCES INFORMATIQUES

ÉPREUVE DU SECOND GROUPE

DURÉE DE L'ÉPREUVE : **20 minutes**

DURÉE DE PRÉPARATION : **20 minutes**

L'utilisation d'une calculatrice est interdit

Le candidat préparera les différentes questions posées dans le sujet. Ces questions portent sur tout ou partie du programme du cycle terminale de la spécialité NSI.

Le candidat pourra répondre directement sur le sujet ou préparer les réponses aux questions sur une feuille de brouillon.

A la fin de la préparation, le candidat se présentera devant le jury muni de ce sujet ainsi que de toutes ses feuilles de brouillon.

Exercice 1 ►

Écrire une fonction `compte_occurrences` prenant en paramètres une valeur `x` et un tableau `tab` (de type `list`) et renvoyant le nombre d'occurrences de `x` dans `tab`. L'objectif de cet exercice étant de parcourir un tableau, il est interdit d'utiliser la méthode `count` des listes Python.

Exemples :

```
1 >>> compte_occurrences(5, [])
2 0
3 >>> compte_occurrences(5, [-2, 3, 1, 5, 3, 7, 4])
4 1
5 >>> compte_occurrences('a', ['a', 'b', 'c', 'a', 'd', 'e', 'a'])
6 3
```

Écrire la fonction `compte_occurrences`

</> Code Python

```
def compte_occurrences(x, tab):
    ''' Compte le nombre d'occurrences de x dans tab.
    x : valeur à compter (int, float, str, etc.)
    tab : tableau de valeurs (list)
    Renvoie : le nombre d'occurrences de x dans tab (int)
    '''

    assert compte_occurrences(5, []) == 0
    assert compte_occurrences(5, [-2, 3, 1, 5, 3, 7, 4]) == 1
    assert compte_occurrences('a', ['a', 'b', 'c', 'a', 'd', 'e', 'a']) == 3
```

Exercice 2 ►

Un coeur de réseau est constitué d'un maillage dans lequel les routeurs R_1 à R_6 sont reliés par des liaisons physiques dont le coût (sans unité) est indiqué sur la figure suivante.

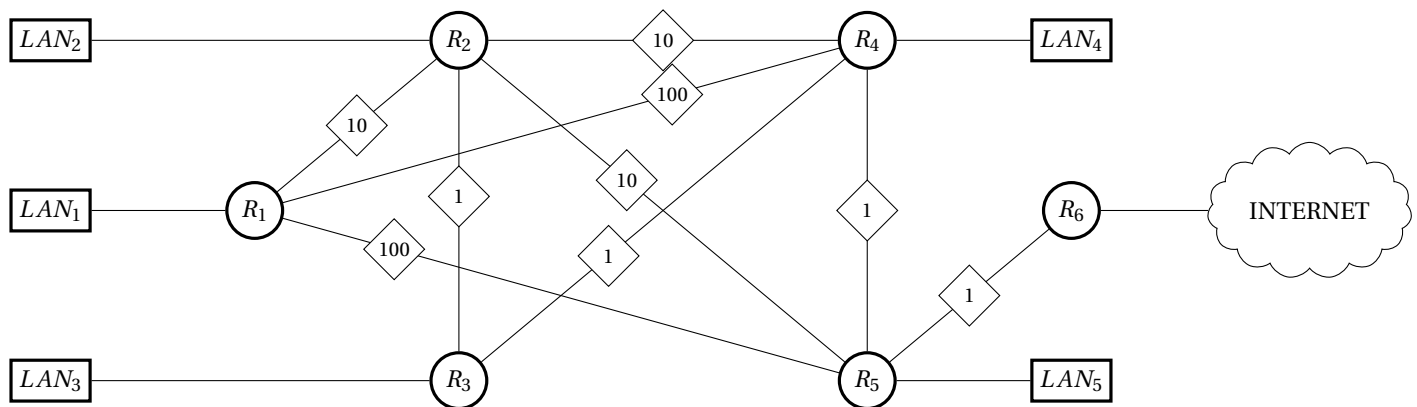


FIGURE 1 – Schéma du réseau

Tableau du coût des liaisons :

Tableau du coût des liaisons		
Routeur 1	Routeur 2	10
Routeur 1	Routeur 4	100
Routeur 1	Routeur 5	100
Routeur 2	Routeur 4	10
Routeur 2	Routeur 3	1
Routeur 2	Routeur 5	10
Routeur 3	Routeur 4	1
Routeur 4	Routeur 5	1
Routeur 5	Routeur 6	1

Derrière chaque routeur R_1 à R_5 , un switch distribue un réseau local LAN_1 à LAN_5 . Le protocole utilisé est le protocole OSPF, qui minimise respectivement la somme des coûts des liaisons physiques empruntées.

1. Compléter la table de routage de R_1 sachant que le protocole de routage OSPF est utilisé.

Un utilisateur du réseau local LAN_1 interroge, via son navigateur web, un serveur d'Internet.

2. Donner, selon le protocole OSPF, la route suivie dans le maillage par une requête partant du réseau LAN_1 à destination d'Internet et en donner la valeur de la distance.

Table de routage R_1		
destination	prochain saut	distance
R_2	R_2	10
R_3		
R_4		
R_5		
R_6		

Le protocole de routage OSPF est toujours utilisé et le routeur R_2 tombe en panne.

3. Donner la nouvelle route suivie dans le maillage par une requête partant du réseau LAN_1 à destination d'Internet et en donner la nouvelle valeur de la distance.

Exercice 3 ►

1. Indiquer le fonctionnement de la pile. Est-ce une structure du type « First-In First-Out » ou « Last-In First-Out »?

Pour implémenter une structure de pile, on a besoin d'un nombre réduit d'opérations de bases :

- `__init__` : méthode renvoyant une pile vide.
- `empiler` : méthode ajoutant un élément `e` sur une pile `p`
- `depiler` : méthode enlevant et renvoyant un élément d'une pile.
- `est_vide` : méthode renvoyant vrai si la pile transmise en paramètre est vide, faux sinon.

2. Indiquer l'état des piles `p1` et `p2` après l'exécution du code suivant :

```

1 p1 = Pile()
2 p2 = Pile()
3 p1.empiler('A')
4 p1.empiler('B')
5 p1.empiler('C')
6 p1.empiler('D')
7 while p1.est_vide() == False:
8     n = p1.depiler()
9     p2.empiler(n)

```

 Code Python

```

class Pile:
    def __init__(self):
        ''' Crée une pile vide qui comporte 0 éléments '''
        self.contenu = []
        self.nb_elements = 0

    def empiler(self, e):
        ''' empile un élément dans la pile '''

    def depiler(self):
        ''' dépile un élément et le renvoie si la pile n est pas vide
        Renvoie None si la pile est vide '''

    def est_vide(self):
        ''' renvoie True si la pile est vide et False sinon '''

#

```

3. Compléter la méthode `est_vide` de manière à ce qu'elle renvoie `True` si la pile est vide et `False` sinon.
4. Compléter la méthode `empiler` qui ajoute un élément à la pile représentée sous forme de liste.
5. Compléter la méthode `depiler` qui supprime un élément de la pile et le renvoie.